

and networking, as well as high level components such as DNS entries, SaaS features, etc. It leaves the configuration management (CM) to tools such as Chef that do the job better. It lays the foundation for automation in infrastructure (both cloud and on-premise) using IaC. Its governance policy makes the cloud operating model system-compliant, which otherwise is only known internally to the IT team.

Terraform is cloud-agnostic and uses a high level declarative style language called HashiCorp Configuration Language (HCL) for defining infrastructure in 'simple for humans to read' configuration files. Organisations can use public templates and can have a unique private registry. Templates are a maintained repository containing pre-made modules for infrastructure components needed under version control systems (like Git).

Installation

Installing Terraform is very simple; just follow the steps mentioned below.

1. Download the archive

```
(https://releases.hashicorp.com/
terraform/${VER}/terraform_${VER}_linux_
amd64.zip):
export VER="0.12.9"
```

```
wget
https://releases.hashicorp.com/
terraform/${VER}/terraform_${VER}_linux_
amd64.zip
```

2. Once downloaded, extract the archive:

```
unzip terraform_${VER}_linux_amd64.zip
```

3. The last step created a Terraform *bin* file in the working directory. For Terraform to be accessible everywhere, move it to *usr/local/bin*:

```
sudo mv terraform /usr/local/bin
```

4. Confirm the Terraform installation:

```
terraform -v //v0.12.9
```

Life cycle

Terraform template (code) is written in HCL language and stored as a configuration file with a *.tf* extension. HCL is a declarative language, which means our goal is just to provide the end state of the infrastructure and Terraform will figure out how to create it. Terraform can be used to create and manage infrastructure across all major cloud platforms. These platforms are referred to as 'providers' in Terraform jargon, and cover AWS, Google Cloud, Azure, Digital Ocean, Open Stack and many others.

Let us now discuss each stage in the IaC lifecycle (Figure 1), which is managed using Terraform templates.

Code

Figure 2 is sample code for starting an EC2 or t2.micro instance on AWS. As visible, only a few lines will instantiate an instance on AWS. It's also implicit that the same code can be maintained under VCS and be used to instantiate instances in various regions with different resource configurations, removing error-prone and time-consuming manual work.

- **Provider:** All major cloud players such as AWS, Azure, GCP and OpenStack have their APIs for

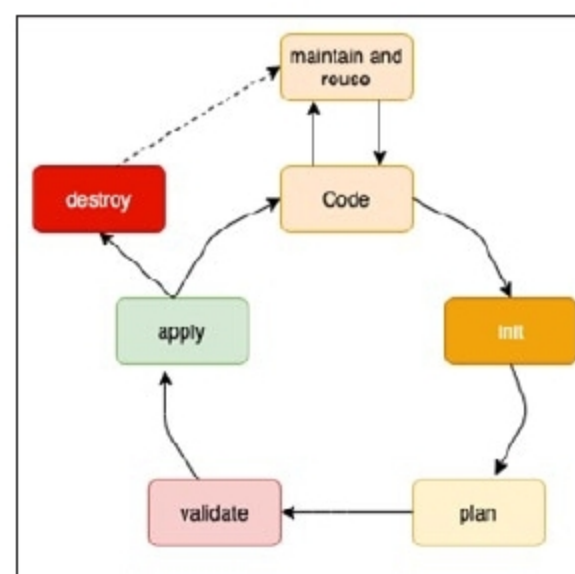


Figure 1: Life cycle of IaC using Terraform

Terraform. These APIs are maintained by the community.

- Username: Key given by provider
- Password: Key given by provider
- Region: Specify region of deployment
- **Resource:** There are many kinds of resources such as an open-stack basic instance, AWS EC2, Droplet and Azure VM, which can be created as follows:
 - resource <provider_instance_type> <identifier>
 - Image ID: This is machine image specific, a tag for an image we need to install (Ubuntu and Windows).
 - Flavour type: This is the type of instance governing the CPU, memory and disk space.

Here, template defines the provider as AWS, and provides the access key, secret key and region for being able to connect to AWS. After that, resource to be create is specified ie. an aws_instance here and is named as "example". Also, the count and instance type (size of instance) is mentioned as code and can be seen in Figure 2.

The same code can be used to set up an instance in another region. Also, Terraform offers users the power of using variables and other logical statements such as if-else and the for loop, which can optimise the setting up of infrastructure even more.

```
1 ## ec2.tf
2 provider "aws" {
3   access_key = "<aws_access_key>"
4   secret_key = "<secret_key>"
5   region = "<aws_region>"
6 }
7 resource "aws_instance" "example" {
8   count = 5
9   ami = "ami-v1"
10  instance_type = "t2.micro"
```

Figure 2: Terraform template code for provisioning an EC2 instance